

Development of a Machine Learning Seizure Detection Algorithm for OpenSeizureDetector

Graham Jones, 24 June 2026

Abstract

A 14 layer, 1 dimensional convolutional neural network (CNN) has been trained on the Open Seizure Database data to produce a useable seizure detection algorithm.

Several combinations of training parameters were investigated using nested k-fold validation to determine an optimum set that gave good seizure detection reliability with small uncertainty in the performance, and a modest improvement in false alarm rate.

- The model developed is giving the same high tonic-clonic seizure detection performance as the original OSD algorithm (>90% sensitivity / True Positive Rate (TPR))
- It is giving improved performance of other (partial/focal) seizures (>80% TPR).
- It is giving a small improvement in false alarm rate.

The phone deployment of the model has been validated by comparing the calculated seizure probabilities to those calculated for the same data on the development computer.

Based on these results, the model is suitable for deployment in the OSD App.

It should be re-trained regularly as more false alarm data is generated to improve false alarm performance.

If it is not possible to improve false alarm performance, alternative neural network structures can be trialled.

Introduction

Open Seizure Detector (<https://openseizuredetector.org.uk>) is a seizure detection and alarm system that utilises a smart watch to detect movement (and possibly heart rate). It analyses the movement and heart rate to detect ‘seizure like’ patterns, and will generate alarms to alert carers that the user may need assistance.

The system contains a ‘Data Sharing’ feature that allows users to contribute their data by uploading the measured data to a central server when the system detects an alarm condition, and the user manually marks this as either a genuine seizure or false alarm.

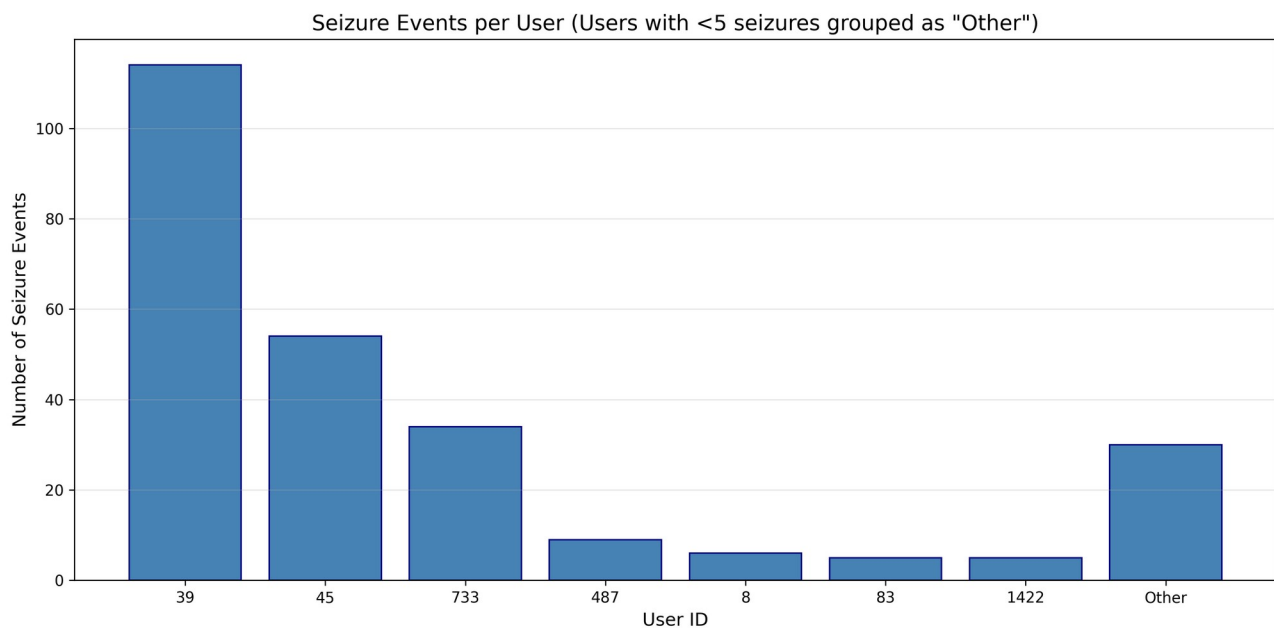
The data is collated periodically and published for research purposes as the Open Seizure Database (OSDB).

This database has been used to train a Machine Learning (ML) model to be incorporated back into OpenSeizureDetector as described in this report.

Input Data

The initial investigations for developing the ML model utilised V1.9 of OSDB.

Final model trained on V1.10 which included more seizures and false alarms. V1.10 contained just over 250 seizure events, distributed between users as shown below:



Model Choices

Several different types of model have been used by researchers for detection of seizures. These range from ‘classic’ machine learning models such as State Vector Machines [1] and Random Forest classifiers [2] to Convolutional Neural Networks [3] and Long-Short Term Memory neural networks [4].

Previous experiments using the Open Seizure Database to train a classic (Random Forest) model had produced results that were comparable to the deterministic OSD Algorithm in terms of True Positive Rate (TPR) but had a similar False Alarm Rate (FAR) so were not advantageous. Therefore for this work a neural network approach was taken.

Options considered were:

- A 'spectrogram' approach where the time dependent spectral power of the movement was converted into an image which could be used in an image classifier.
- An LSTM based classifier similar to that described in [4].
- A 1-dimensional Convolutional Neural Network (CNN-1d) that analyses raw accelerometer magnitude data.
- A 3-dimensional Convolutional Neural Network (CNN-3d) that analyses raw accelerometer axes data.

It is preferred to use a model that requires minimum computation in operation to limit battery consumption, so the 1-dimensional CNN (CNN-1d) approach was selected. This had initially been discounted because it was expected to suffer from over-fitting, but very good results were reported in [3]. This also has the advantage of not requiring heart rate data for the classification (unlike the approach in [4] – a pure accelerometer solution is preferred because the PineTime watches preferred by OpenSeizureDetector have less reliable heart rate measurement than more expensive devices.

The intention is that if the results of the CNN-1d model are not sufficient to meet the objective, it will be extended to 3 dimensions to attempt to improve performance. If this is also unsuccessful the spectrogram and LSTM models will be tried next.

Training Methodology

A significant concern with switching to a Machine Learning algorithm rather than a deterministic one is that it may be possible to train the Machine Learning model to recognise the specific seizures in the database, but it may not be general enough to recognise seizures from other users.

For this reason a multi-layer approach to training was adopted that will allow as much data to be used for training the model as possible (to improve its accuracy), while allowing it to be tested on data that has not been used in the training process to estimate its 'real world' performance.

Splitting Data

The data was split using a nested k-fold approach, whereby all of the available data was split into three equal sets, and three outer-fold iterations were carried out with each of the sets being kept back as an independent test set. The other two sets were split into three parts and k-fold validation used to train three different models (with one of the three folds being used for validation to select the best model during training).

This results in 9 models being trained (3 sets of three), with one third of the total dataset being available for independent testing of the resulting models.

Data Augmentation

Two different types of augmentation were applied to the seizure data to reduce the imbalance between seizure and non-seizure events.

- Phase Augmentation – the start of each data sample was moved by a selected step to create a new sample of similar data, but offset by a small period.
- Noise Augmentation: Gaussian noise was added to each measurement to create a new sample of data. The noise augmentation factor is the number of augmented events created from each real seizure event.

Several model training runs were carried out to determine the optimum augmentation that would give a good TPR without increasing FPR.

Choosing the ‘Best’ Model during Training

Every 5000 training iterations the validation dataset was evaluated on the model and statistics calculated. Most neural network training schemes would consider the ‘loss’ or ‘accuracy’ statistic and save the model if the chosen statistic improves compared to previous iterations.

The seizure database is very imbalanced, so it is possible to calculate a large accuracy even if the true positive rate is very small, so this is not a particularly helpful statistic for model saving.

Spahr et. al. [3] reported good results based on saving the model if the Sensitivity (True Positive Rate, TPR) and False Alarm Rate (FAR) both improve, or if the FAR improves by more than 10% and the TPR is within 5% of the previous best model.

This approach was found to sometimes ignore models which appeared to be good (a low FAR, with TPR >25%). It would also select some models with a very high TPR, even though FAR is relatively high). To avoid this a minimum TPR is specified so the model will not be saved if the TPR exceeds that threshold, and the model with the minimum FAR was selected.

Testing Methodology

Validation during Training

During the automated training, validation is only used to select the best model during training iterations – other optimisations (level of augmentation etc.) were done manually by re-running the training with different settings and comparing the results.

Independent Testing after Training

Once the specified number of training iterations were completed, the best of the three models developed in the inner fold training was tested against the outer fold test data which was kept back.

This gives our best estimate of the expected performance of the model in the real world, as the outer fold test data has not been used in any way in the training and selection of the model (the inner fold

validation data set is less suitable because the model was selected based on the results from that data, so it is not independent, so we would expect good results from the validation data set).

The final model was selected as the best of the three outer loop models.

Verification of Mobile Implementation

The implementation of the algorithm on the phone uses the 'execuTorch' library [7] which requires the model to be converted from the normal .pt pytorch version to a '.pte' format which includes some simplification and quantisation.

To confirm that this conversion had worked correctly the independent test dataset was used to test both the original .pt and the converted .pte model, and the results compared. It was found that the results for the .pt and .pte model only varied in the third or fourth significant figure of seizure probability, so the differences were insignificant, confirming that the .pte model is valid.

Finally it is possible that the java implementation of the data processing and model inference could be different to the python version used during development. A simple script was developed that fed the independent test data into the mobile implementation on the phone via the web server that is built into the mobile app, and retrieved the alarm condition and seizure probability for every datapoint in the test data. The seizure probabilities calculated on the mobile app and on the development computer were compared for each datapoint. There was no detectable difference in the calculated seizure probability (to 4 decimal places of probability), so the mobile implementation is correctly determining the seizure probability.

Real-World Testing

A beta test version of the OpenSeizureDetector Android App [5] was prepared which included the ability to select and download pytorch neural network models from a web site. This allowed various models to be trialled without having to re-build and re-distribute the Android phone app.

The model was tested by wearing a PineTime watch, with the OpenSeizureDetector firmware installed [6]. The testing using the OpenSeizureDatabase seizure data had demonstrated that the model is reliable at detecting previously unseen convulsive seizures, so the purpose of the real-world testing is to identify causes of false alarms and log false alarm data so the model can be re-trained to reduce the false alarm rate (FAR).

Results

Parameter Optimisation

The initial development of the model was carried out on Version 1.9 of the OpenSeizureDatabase, with several training runs being carried out (each using 3x3 nested k-fold validation) to determine the TPR and FAR with uncertainty for different parameters. The main runs are summarised in Table 1 and plotted in Figure 1. The figure and table present 'per event' TPR and FPR where a true positive result is obtained if any of the data points within a seizure event are identified as being a seizure, and a false positive is obtained if any of the data points within a non-seizure event are identified as being a seizure.

It can be seen that Runs 14-18 were giving a reasonable TPR for all seizures of around 80%, and a better FPR than obtained from the deterministic OSD algorithm. However, the performance for tonic-clonic seizures was worse than the original OSD algorithm.

Run 21 (no augmentation) gave relatively good TPR, but the FPR was significantly higher than the OSD algorithm, so this was unsuitable. It was also noted that the uncertainty in the TPR was quite high, probably because the min TPR and max FPR were too restrictive so the model was not selecting the true 'best' model but using a fallback approach to select a model during training. Therefore for Run 22 and Run 23, the TPR/FPR constraints were relaxed so that we could obtain more repeatable results, which can be seen from the lower uncertainties on TPR.

It was considered that the Run_23 results were good, because of the relatively high TPR with lower uncertainty on TPR. The Run_23 parameters were therefore retained and the model trained on more data from the Version 1.10 of the database, to give Run_24. Run_24 gave more like the expected performance with Tonic Clonic seizure detection being better than all seizures, and gave a modest improvement in FPR compared to the OSD algorithm. Note that Run_24 was trained using a standard train/test/validation data split of 70/20/10 distribution (split by whole seizure event) which will be used for 'production' training. This does however mean that the uncertainty in the TPR is not calculated (because that requires k-fold validation) – the uncertainty would however be expected to be similar to Run_23 because it used the same training parameters.

The results of testing on the 20% test dataset were analysed in detail on a 'per-event' basis as shown in Figures 2 and 3. Of the 7 seizures (out of 43) that were not detected, three of them had comments such as "possibly prelude to this mornings other ones – check data", "very short, not sure if it was real or not", "not sure if this is a real one or not", "focal", which implies that we may not really expect the algorithm to detect these. Two out of the 22 tonic-clonic seizures were not detected, which gives the >90% TPR calculated.

Given the good TPR results against previously unseen data, it appears that the Run_24 model is suitable for deployment, although it gives only a small FAR improvement at the moment.

Model Verification

The .pt pytorch model was converted into a .pte model for use in the mobile ExecuTorch library, and the .pte model used to determine the seizure probabilities for the test dataset.

The TPR and FPR results were compared on a per-data point and a per-event basis and the numbers were identical. The calculated seizure probabilities were compared and it was found that they only differ in the third or fourth significant figure.

The model was deployed on the OpenSeizureDetector android app, and the same independent test data fed into the app - the android implementation returned identical seizure probabilities (after the first 30 seconds that it takes to fill the buffer) – Figure 4.

Final Testing

The Run24 model was deployed during the beta testing phase of Version 5 of OpenSeizureDetector. Some initial problems were identified with the .pte model not working on older CPUs (because it uses certain optimisations for efficiency), so the app was modified to check that the CPU optimisations were present on the device before trying to run the model.

In the ~6 months since the Run24 model was trained, more seizures have been contributed to the OpenSeizureDatabase (but not published at the time of writing). These new seizures were used to test the on-device deployment of the model. On-device testing is important because the Android app does not generate an alarm for every model analysis that give >50% seizure probability. By default it needs to see 2 consecutive periods giving >50% probability to give a WARNING alert and a further period with >50% probability to give a full ALARM activation. It is to be expected therefore that the true positive rate when deployed will be lower than that calculated on a by-event basis reported above.

To do this final testing the following process was followed:

- Removed entries with bad data (ie no discernable change in movement or heart rate at reported time, or very few datapoints).
- Removed duplicates where times are very close together so are essentially analysing the same data twice.
- Filtered only for events with id >900000, which corresponds to after the release of V1.10 of OSDB, so none of these events have been used in the training of the run24 model.
- Run the remaining events through the deployed model by setting the app to 'Garmin' data source mode and using http to inject the data from the database rather than using a watch.

The analysis of the results gave a surprisingly poor true positive rate (TPR) - 56% for all seizures, and 58% for tonic-clonic.

The reason for the poor result was a lot of very recent data contributed by a single user (#1643). These seizures appear to be unusual – often very short and do not have the usual shape of a TC seizure, but they do have a strong heart rate response. On some of the events the user marked it as 'non-epileptic seizure'. Given the uncertainty over whether these seizures are actually tonic-clonic seizures or something else, they were removed from the dataset pending further investigation.

After removing data from user #1643, we have 30 seizures contributed by 5 users, with 83% TPR.

The by user results are:

- User 39: 19 out of 20 correct (95% TPR).
- User 733: 4 out of 4 correct (100% TPR)
- User 1246: 2 out of 4 correct (50% TPR)
- User 1422: 0 out of 1 correct (0% TPR). Generated a warning, user note says not sure how strong the seizure was, so maybe not a TC? Very subtle movement, but big HR response though
- User 1640: 0 out of 1 correct (0% TPR). Generated a warning. Lot of movement and big HR response, so surprised ML model did not generate alarm.

Given the above results it can be seen that the deployed model is giving a reasonable TPR of over 80%, although for some users the performance may be significantly worse than that. Therefore in

Version 5 of OpenSeizureDetector, the current deterministic algorithms will be retained so that users can gain confidence in the performance of the ML model while using the deterministic models as a backup.

Conclusions

- The model developed is giving the same high tonic-clonic seizure detection performance as the original OSD algorithm
- It is giving improved performance of other (partial/focal) seizures.
- It is giving a small improvement in false alarm rate.
- The phone deployment of the model has been validated by comparing the calculated seizure probabilities to those calculated for the same data on the development computer.
- The model has been tested on a whole system basis (including generation of warnings before alarms) and achieved a True Positive Rate of over 80%, on data that has not been used at all during the training and validation.
- Based on these results, the model is suitable for deployment in the OSD App.
- It should be re-trained regularly as more false alarm data is generated to improve false alarm performance.
- If it is not possible to improve false alarm performance, alternative neural network structures can be trialled.

References

- [1] – Onorati, F., Regalia, G., Caborni, C., et al. (2017). Multicenter clinical assessment of improved wearable multimodal convulsive seizure detectors. *Epilepsia*, 58, 1870-1879. <https://doi.org/10.1111/epi.13899>
- [2] – Vakilna, Y. S., Li, X., Hampson, J. S., et al. (2024). Reliable detection of generalized convulsive seizures using an off-the-shelf digital watch: A multisite phase 2 study. *Epilepsia*, 65, 2054–2068. <https://doi.org/10.1111/epi.17974>
- [3] – Spahr et. al.; “Deep learning-based detection of generalized convulsive seizures using a wrist-worn accelerometer”; *Epilepsia*. 2025;66(Suppl. 3). <https://doi.org/10.1111/epi.18406>
- [4] – Pordoy et. al.; “Enhanced Non-EEG Multimodal Seizure Detection: A Real-World Model for Identifying Generalised Seizures Across the Ictal State”; *IEEE J Biomed Health Inform.* 2025 May;29(5):3329-3342. doi: 10.1109/JBHI.2025.3532223. Epub 2025 May 6. PMID: 40031183.
- [5] – https://github.com/OpenSeizureDetector/Android_Pebble_SD – OpenSeizureDetector Android App.

[6] - <https://openseizuredetector.github.io/pages-user/pine-time-installation.html> – PineTime watch installation instructions.

[7] - <https://docs.pytorch.org/executorch/stable/index.html> – ExecuTorch mobile neural network inference library.

Figures

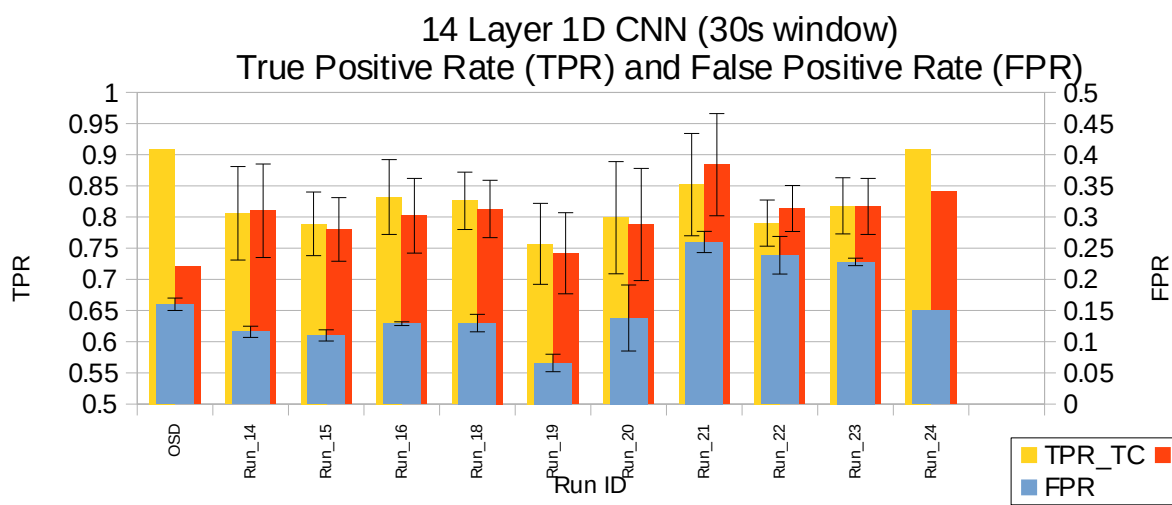


Figure
Parameter Optimisation Runs

1: Results from

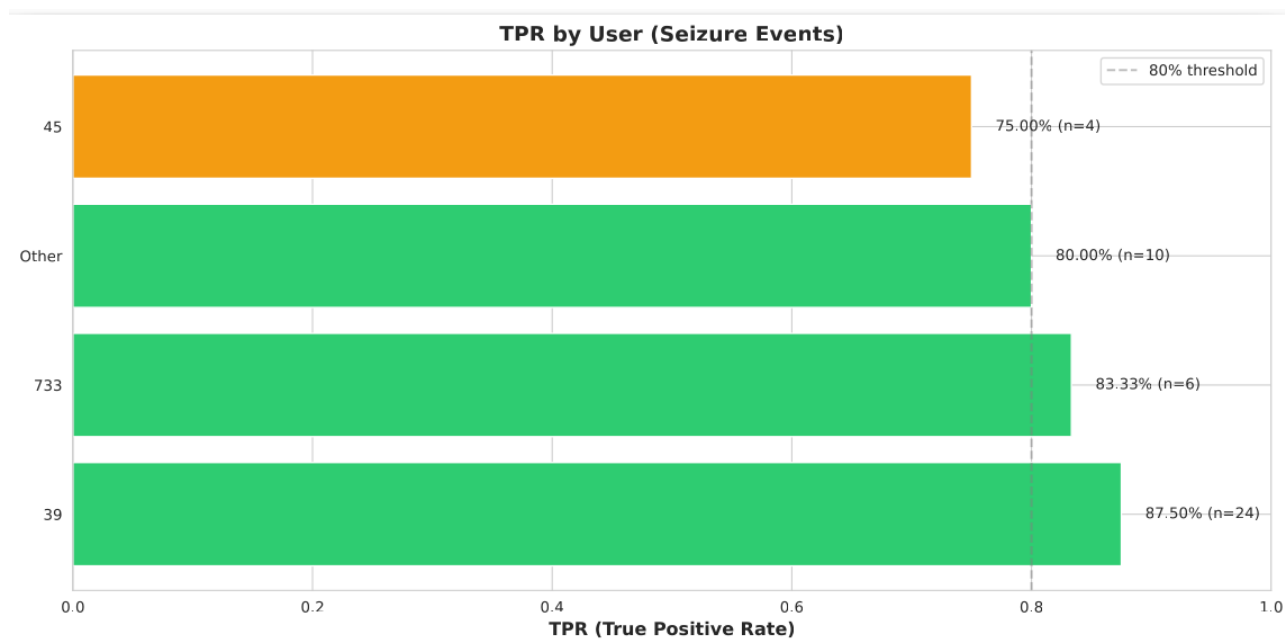


Figure 2: Run_24, All Seizures, TPR by User

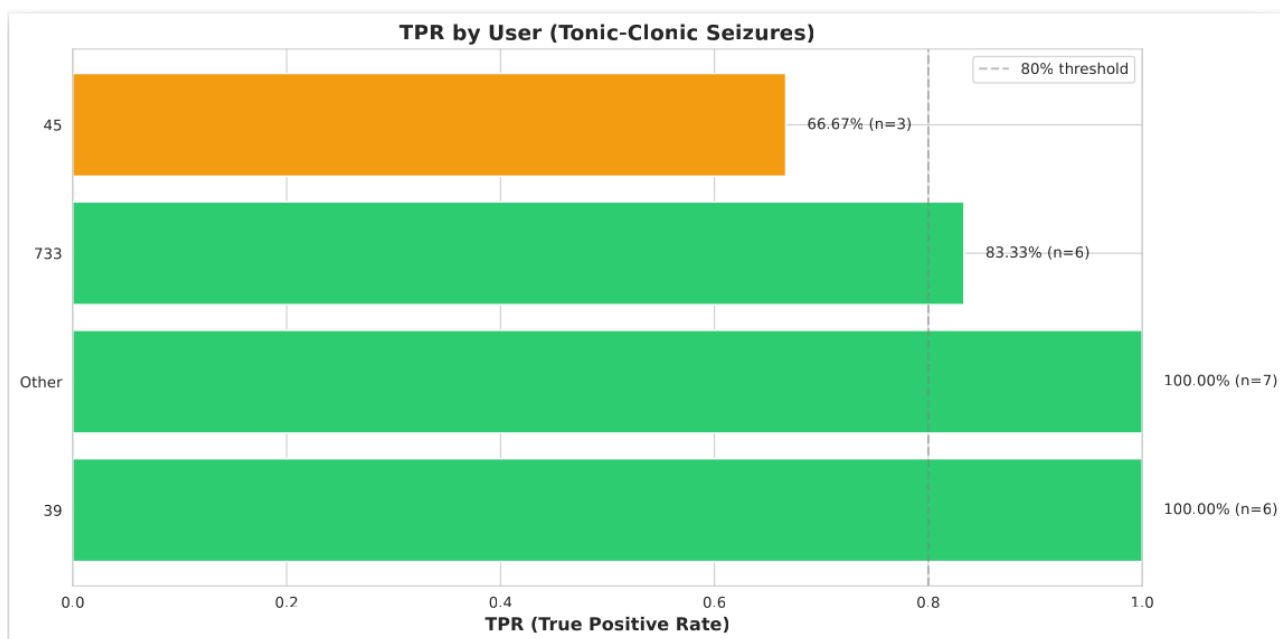


Figure 3: Run_24, Tonic-Clonic Seizures - TPR By User

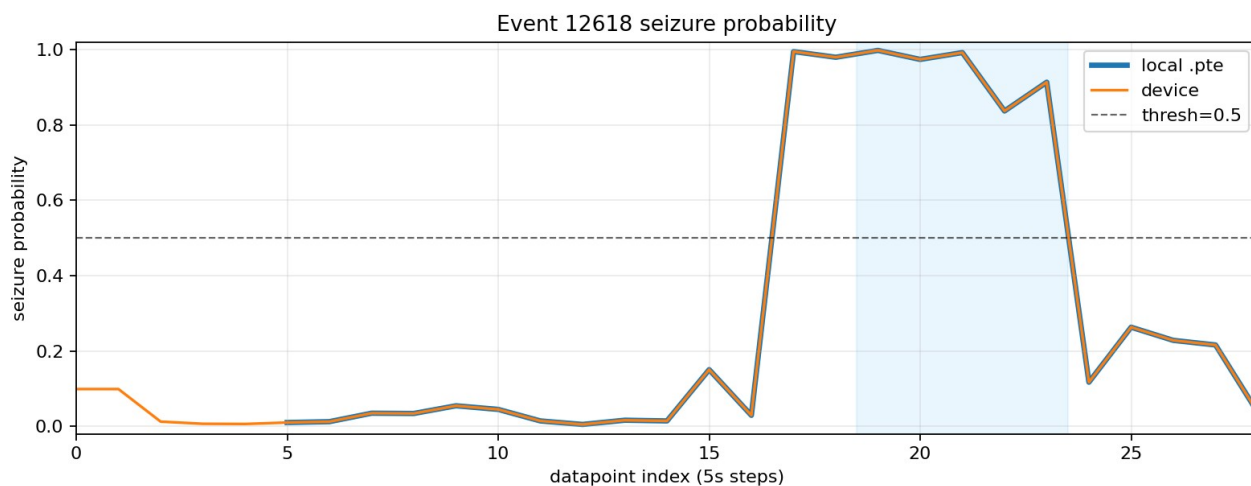


Figure 4: Seizure Probabilities calculated in the Android App ('device') and the .pte on the development computer ('local .pte'). The shaded area shows when the app generated an ALARM condition.

Tables

Figure 5: Results of Parameter Optimisation Runs (As plotted in Figure 1)

Run	Description	Min TPR	Max FAR	Phase Aug.	Noise Aug.	TPR	stdev (TPR)	TPR TC	stdev (TPR TC)	FPR	stdev (FPR)
OSD	Deterministic OpenSeizureDetector Method for comparison					0.72	0.02	0.81		0.18	0.01
Run_14	3x3 kFold, All Users (V1.9), only noise aug.	0.25	0.015	none	10 x 30 milli-g	0.81	0.026	0.81	0.075	0.12	0.009
Run_15	3x3 kFold, All Users (V1.9), phase aug and noise aug	0.25	0.015	Step=5	10 x 30 milli-g	0.78	0.037	0.79	0.051	0.11	0.009
Run_16	3x3 kFold, All Users (V1.9), only noise aug, increased min. TPR	0.45	0.030	none	10 x 30 milli-g	0.80	0.025	0.83	0.060	0.13	0.003
Run_18	3x3 kFold, All Users (V1.9), excluding auras, only noise aug	0.45	0.030	none	10 x 30 milli-g	0.81	0.026	0.83	0.046	0.13	0.014
Run_19	3x3 kFold, All Users (V1.9), excluding auras, extra noise aug.	0.45	0.030	none	30 x 30 milli-g	0.74	0.069	0.76	0.065	0.07	0.014
Run_20	3x3 kFold, All Users (V1.9), excluding auras, extra noise aug.	0.45	0.030	none	10 x 90 milli-g	0.79	0.043	0.80	0.090	0.14	0.053
Run_21	3x3 kFold, All Users (V1.9), excluding auras, no augmentation	0.45	0.030	none	none	0.88	0.052	0.85	0.082	0.26	0.017
Run_22	3x3 kFold, All Users (V1.9), excluding auras, no augmentation, more relaxed thresholds	0.40	0.045	none	none	0.81	0.063	0.79	0.037	0.24	0.030
Run_23	3x3 kFold, All Users (V1.9), excluding auras, noise aug, more relaxed thresholds	0.40	0.045	none	10 x 30 milli-g	0.82	0.078	0.82	0.045	0.23	0.006